Solution architecture — TrustBank CBS (.NET 10)

Every project in the solution, why it exists, what it does, how they depend on each other, and how a request flows through them.

- **Solution root:** `E:\Adhir\AdWork\TrustBank.Code\TflCbsNet10Sol\`
- **Solution file:** `TflCbsNet10Sol.slnx` — **36 projects**.
- **All projects target** `net10.0` (set centrally in `Directory.Build.props`), with Central Package

Management (versions in `Directory.Packages.props`).

Also on disk, deliberately outside the `.slnx`: `TflCbs.Lab` / `TflCbs.Lab.Demo` (the proc-conversion lab — experimental, API-drifted) and `TflCbs.Tools.PerfProbe` (a measurement tool run directly with `dotnet run`). Plus two **vendored in-house library trees** with their own solutions — `libs/TflOmniDb/` and `libs/TflSecurityCrypto/` (§3) — the `TflCbs.E2E` Playwright suite (a Node project, not a .NET one), and the gitignored `codebase_shared/` handover tree (§4).

> **Last reviewed:** 2026-08-21. The modular-monolith split completed on 2026-07-29 (`TflCbsServices` dissolved); the in-house libraries were vendored into the repo on 2026-08-19; the canonical config files moved to solution-level `config/` on 2026-08-21.

---

1. The big picture (why this shape)

This is an incremental **strangler-fig** migration of TrustBank CBS (core banking) from ASP.NET WebForms / VB.NET / .NET 4.8 to .NET 10 / C# MVC.

Two architectural decisions drive the project layout:

- **A. HORIZONTAL split (framework vs screens).** Reusable web plumbing lives in a framework library

that knows NOTHING about individual screens OR about any business module; screens depend on the framework, never the reverse. Compiler-enforced (the framework references only `TflCbs.Abstractions`) and arch-tested.

- **B. VERTICAL split (MODULAR MONOLITH).** The business layer is divided into per-domain bounded

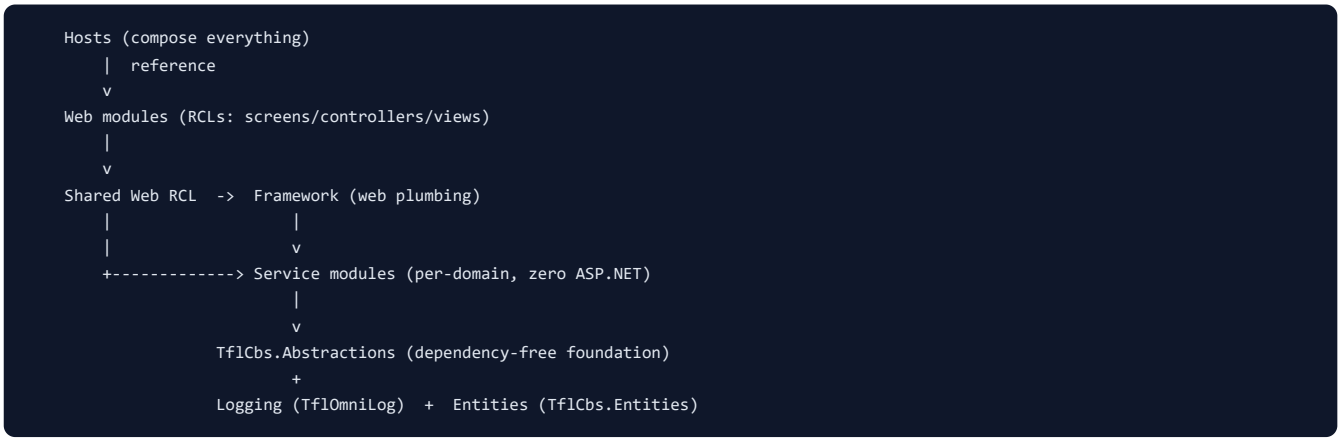
modules, each its OWN ASSEMBLY. A module is a bounded domain, never a per-service assembly. Steady state is ONE process + ONE database, but each domain can also be hosted alone (thin hosts) behind a gateway — which is what enables cross-host SSO.

Decision B is **complete**. The old single business assembly `TflCbsServices` no longer exists: all 8 domains were promoted to their own assemblies and the composition contracts moved down into `TflCbs.Abstractions`. Two project names still carry the historical prefix (`TflCbs.Tests` / `.Demo`) — they test and exercise the module assemblies, not a project called `TflCbsServices`.

The result: a small host composes independently-buildable pieces. The data layer never references ASP.NET; screens never reach past the framework; the framework never reaches into a module.

---

2. Layers at a glance (dependency direction is strictly INWARD)



Rule: hosts → web modules → framework/shared → service modules → abstractions → logging/entities.

`TflCbs.Abstractions` is the **FLOOR**. Everything above it references it; it references nothing but two `Microsoft.Extensions.*.Abstractions` packages. No `TflCbs.Entities`, no `TflOmniDb`, no ASP.NET. Keep it that way — it is what lets the framework and the modules share contracts without seeing each other.

The framework references **ONLY** `TflCbs.Abstractions` (+ `TflOmniLog` / `TflOmniDb`). It CANNOT reference a service module — that is a compile error, not a convention. Communication runs the other way through the four framework **PORTS** in `TflCbs.Abstractions/FrameworkPorts.cs` (`IBroadcastSource`, `IBankVariableSource`, `IMenuSource`, `ISessionValidator`): the framework declares the interface, a core module implements it, the host wires them up. With no implementation present the framework falls back to no-op defaults, so a host runs (degraded) without General or Authentication.

---

3. External / vendored dependencies

Two in-house libraries are consumed as **bare DLLs by** `HintPath` rather than `ProjectReference` — but since **2026-08-19 their source lives inside this repo**, so a fresh clone builds without any folder outside the solution root. Before that they sat two levels above it under `_Archive\Ad.ProjectLibrary\`, which is why the reference style is a HintPath at all.

Library	Source in repo	Consumed from	What it is
`TflOmniDb.dll`	`libs/TflOmniDb/TflOmniDb/`	`..\libs\TflOmniDb\TflOmniDb\bin\Debug\net10.0\`	Generic multi-provider DAL: `Repository<T>`, `DataAccess`, `IUnitOfWork`, SQL Server / Oracle / PostgreSQL dialects. Editable in-house SOURCE, not a black box — it also ships a scaffolder, a migration tool, benchmarks and a demo (own solution under `libs/TflOmniDb/`).
`TflSecurityCrypto.dll`	`libs/TflSecurityCrypto/TflSecurityCrypto/`	`..\libs\TflSecurityCrypto\TflSecurityCrypto\bin\Debug\net10.0\`	PBKDF2 password storage plus a byte-for-byte port of the legacy VB.NET crypto helpers (used by login).

`TflCbs.Entities.dll` is referenced the same way (`..\TflCbs.Entities\bin\Debug\net10.0\`) but IS a solution project — see §4. `TflOmniLog` is a first-party solution project, referenced normally.

Consequence of the bare-DLL style: these references and the ADO.NET drivers do **not** flow transitively, so every project in the closure re-declares the ones it needs.

---

4. Every project, by layer

Layer: foundation (dependency-free)

`TflCbs.Abstractions` — class library, ns `TflCbs.Abstractions`

- **Purpose.** The shared floor every module and the framework compile against. Holds: `Result` / `Result<T>` (the service return contract); `ICbsModule` / `ICoreCbsModule` + `AddCbsModules` (composition and discovery — `ModuleRegistration.cs`); the FOUR framework ports (`FrameworkPorts.cs`); the boundary DTOs `MenuListRow` / `BranchListItem` (`MenuModels.cs`); and the shared service helpers `ServiceQuery` (`Has` / `Eq` / `Val` / `Paged` + page-size bounds) and `DbErrors` (`IsDuplicate` / `IsForeignKey`).

- **Refs.** `Microsoft.Extensions.DependencyInjection.Abstractions` + `.Configuration.Abstractions`.

NOTHING else — no `TflCbs.Entities`, no `TflOmniDb`, no ASP.NET, no project references.

- **Rule.** NEVER re-declare a service helper privately. Consume via

`using static TflCbs.Abstractions.ServiceQuery;` / `…DbErrors;`. These were copy-pasted into 8 services before extraction, so `SharedServiceHelperTests` now FAILS THE BUILD on any private copy of `Has` / `Eq` / `Val` / `Paged` / `IsDuplicate` / `IsForeignKey` / `DefaultPageSize` / `MaxPageSize`. Different semantics? Different name.

`TflCbs.Modules.General.Contracts` — class library

- **Purpose.** General's PUBLISHED cross-module surface, so a consumer can depend on General's

contract without referencing General's implementation. Two interfaces: `IScrollService` + `ScrollSources` (the maker-checker scroll) and `IProcessBlockCheck`.

- **Consumed by.** `TflCbs.Modules.RetailBanking`, `TflCbs.Core.Authentication`.
- **Refs.** `TflCbs.Abstractions`; bare-DLL `TflOmniDb`.

Layer: entities

`TflCbs.Entities` — class library

- **Purpose.** Auto-generated persistence entities (`[DbTable]` / `[DbColumn]`, **UPPERCASE** nullable

props, varchar→`AnsiString`). The DB-shape contract every data/business project binds to.

- **Refs.** bare-DLL `TflOmniDb` only.
- **Note.** Consumed by HintPath, but every consumer also carries a

`ProjectReference … ReferenceOutputAssembly="false"` **build-order edge** (added 2026-08-21), so a single solution build — CLI or Visual Studio's Rebuild All — sequences it correctly. The old "build `TflCbs.Entities` first by hand" discipline is gone.

Layer: logging (reusable, ASP.NET-free)

- **`TflOmniLog`** — Serilog-backed logging (`ITflLogger`): file/console/SQL sinks, category logging,

correlation, PII masking, audit, perf timing. Kept ASP.NET-free so the zero-ASP.NET data layer can use it. Refs: none (Serilog + `Microsoft.Extensions.*` abstractions).

- **`TflOmniLog.Demo`** — console tool exercising `TflOmniLog`.
- **`TflOmniLog.Tests`** — xUnit unit tests for `TflOmniLog`.

Layer: service modules (per-domain, zero ASP.NET)

Every module is its own assembly, namespace == assembly name, and each is an `ICbsModule` that self-registers its services. Uniform reference set: `TflCbs.Abstractions` + bare-DLL `TflCbs.Entities` / `TflOmniDb` + the two `Microsoft.Extensions` abstractions packages. Exceptions are called out per project.

CORE MODULES (`ICoreCbsModule` — ALWAYS registered, never gated by config):

Module	Purpose	Extra refs
`TflCbs.Modules.General`	The platform module. Owns three of the four framework ports plus the cross-cutting mechanisms: menu (`MenuListService`), bank variables (`BankVariableService`), broadcast + alerts (`AlertService`), the maker-checker scroll (`ScrollService`), the process-block check, and `GeneralService` / `GeneralGrp11Service`.	`General.Contracts` (it implements them), `TflOmniLog`
`TflCbs.Modules.Reference`	Shared master data with no domain behaviour: geography — `StateService`, `DistrictService`, `TalukaService`.	`TflOmniLog`
`TflCbs.Core.Authentication`	Login/credential + session validation in its own assembly (the first Phase-B split), independent of General. `AuthenticationService`, `ConnectedUserService`, `ShellSessionService`, `LoginRules`, `PasswordPolicyRules`, and `LoginInfo` — the ONLY sanctioned parser of `b_UserLogTime.LoginInfo`. Implements `ISessionValidator`. `ICbsModule.Name = "Authentication"`.	`General.Contracts`; bare-DLL `TflSecurityCrypto`

> **Admission rule for `Reference` :** a master belongs there only if it has NO domain behaviour (pure CRUD + lookup) **and** ≥2 domains consume it. Geography passes; `LockerMake` does NOT (Lockers-only). Without that test this project becomes a dumping ground.

DOMAIN MODULES (gated by `Modules:Enabled`; all promoted 2026-07-29):

Module	Services	Notes
`TflCbs.Modules.HR`	`DiscipActionHistoryService`	Module name is `HR` (UPPERCASE) everywhere — module class, `Modules:Enabled` in every host, and `compose.multi.yaml`. The web RCL stays `TflCbs.Modules.Hr.Web` and routes stay `/Hr/…`; assembly and URL identifiers are a separate axis from the module name.
`TflCbs.Modules.Lockers`	`LockerTypeService`	The Phase-B template and the first fully isolated vertical slice.
`TflCbs.Modules.Accounts`	`BusinessAssessmentService` (+ models)	
`TflCbs.Modules.Clearing`	`InwardClearingService` (+ models)	
`TflCbs.Modules.Administration`	`ModuleService`, `RoleService`	
`TflCbs.Modules.RetailBanking`	`AccountService`, `HoldingAmountService`	+ `General.Contracts` (for `IScrollService`)

Layer: framework / web plumbing

`TflCbs.Framework` — class library, ns `TflCbs.Framework.*`

- **Purpose.** Reusable web infra with one-call setup `AddCbsFramework()` + `UseCbsFramework()`:

session/cache, `CbsAccessMiddleware` (fail-closed guard), the menu facade, the reusable search / record picker / account picker / date picker support, opaque `RowToken`, `UrlNormalizer`, multi-provider `DataAccess` / `DatabaseSelector` wiring, the generic `IReferenceCache` (Redis-optional, fail-open), health checks, security response headers, forwarded headers, global antiforgery, the `cbs-login` rate-limit policy (so every host throttles credential POSTs, not just the gateway), OpenTelemetry (`AddCbsObservability`), and SSO (CbsAuth cookie + shared DataProtection key ring + revocation). Owns ADO.NET driver wiring.

- **Refs.** `TflCbs.Abstractions`, `TflOmniLog`; bare-DLL `TflOmniDb`; `FrameworkReference

Microsoft.AspNetCore.App`. Packages: SqlClient, Oracle, Npgsql, Redis + SqlServer caching, OpenTelemetry (hosting, OTLP exporter, ASP.NET Core / HTTP / runtime instrumentation).

- **Note.** NO reference to any service module or screen RCL — compiler-enforced and arch-tested.

`internal`: `CbsAccessMiddleware`, `UrlNormalizer`. Everything else public. Extension methods live in `Microsoft.Extensions.DependencyInjection` / `Microsoft.AspNetCore.Builder` (the one namespace exception).

Layer: shared web RCL

`TflCbs.Web.Shared` — Razor Class Library, NO controllers

- **Purpose.** Shared layout + shared partials (`_RecordPicker`, `_DatePicker`, `_AccountPicker`,

`_SearchModal`, `_Notification`, `_ActionBar`, `_DbSwitcher` …), root `_ViewImports` / `_ViewStart`, and ALL shared static assets served from `~/_content/TflCbs.Web.Shared/` (css/img/lib/site.js/cbs-*.js).

- **Refs.** `TflCbs.Framework`; bare-DLL `TflOmniDb` (for `_DbSwitcher`).

Layer: core web modules (RCLs, ALWAYS ON)

Project	Marker	Purpose	Refs
`TflCbs.Core.Authentication.Web`	`AuthenticationWebModuleMarker`	Login/logout web module (`AccountController` + Login view). Every host registers it; in multi-host one host is login authority.	Web.Shared, Framework, `Modules.General`, `Core.Authentication`, `TflCbs.Entities` (proj); bare-DLL `TflSecurityCrypto`
`TflCbs.Core.Shell.Web`	`ShellWebModuleMarker`	The non-auth shell every host needs: Home/menu, Modules picker, Components (dev showcase), AccessDenied, Error.	Web.Shared, Framework, `Modules.General`, `Core.Authentication` (password-expiry alert), `TflCbs.Entities` (proj)

Layer: domain web modules (RCLs, gated by `Modules:Enabled`)

All five are Razor Class Libraries, each with an `<Area>WebModuleMarker` for `AddApplicationPart`. Each references Web.Shared + Framework + `TflCbs.Entities` (proj), PLUS exactly the service module(s) it consumes — no bare DLLs, and no reference to a module it does not use. **Only hosts that ENABLE**

a module reference it, so a thin host no longer ships other domains' code.

RCL	Area	Screens	Service-module refs
`TflCbs.Modules.Bank.Web`	`Bank`	ReferenceCache admin (`/Bank/ReferenceCache`).	**None** — gated on the `General` service module (the one Area vs. service-module name divergence), but `IReferenceCache` lives in the framework.
`TflCbs.Modules.Hr.Web`	`Hr`	The geographic masters State/District/Taluka + DiscipActionHistory.	`Modules.Reference` , `Modules.HR`
`TflCbs.Modules.Lockers.Web`	`Lockers`	Locker screens + own `locker-type.js` .	`Modules.Lockers`
`TflCbs.Modules.RetailBanking.Web`	`RetailBanking`	Account / Clients / AccHoldingAmount + `acc-holding-amount.js` .	`Modules.RetailBanking`
`TflCbs.Modules.Administration.Web`	`Administration`	Module + Role masters and ConnectedUsers (`/Administration/Module` , `/Role` , `/ConnectedUsers`).	`Modules.Administration` , `Core.Authentication`

Layer: hosts / composition roots

`TflCbs.Host.Main` — ASP.NET web app, **THE PRIMARY/DEFAULT HOST**. Dev `http :5019` / `https :7225` .

- **Purpose.** Thin MVC host + composition root: `Program.cs` wiring, health checks, and two utility

controllers (`DevController` , `KeepAliveController`). Registers core web parts always, domain web parts gated by `Modules:Enabled` (which is `"*"` here). Also the login authority behind the gateway. Its own `Views/` and `wwwroot/` are empty but for `_ViewImports.cshtml` .

- **Refs.** all five `Modules.`*`.Web`* + *both* `Core.` `.Web` + Web.Shared + Framework + ALL NINE service

modules (General, Reference, Core.Authentication, Lockers, HR, Accounts, Clearing, Administration, RetailBanking) + `TflOmniLog` + `TflCbs.Entities` (proj); bare-DLL `TflCbs.Entities` / `TflOmniDb` / `TflSecurityCrypto` ; SqlClient/Oracle/Npgsql.

- **Canonical config is NOT here any more** (moved 2026-08-21) — see the `config/` note below.

Thin per-module hosts — same `Program.cs` shape, one domain each:

Host	Ports (http/https)	Composes
`TflCbs.Host.Hr`	5300 / 7300	`Modules.Hr.Web` + core web RCLs + framework + General/Reference/Core.Authentication/HR
`TflCbs.Host.Lockers`	5200 / 7200	`Modules.Lockers.Web` + `Modules.Lockers`
`TflCbs.Host.RetailBanking`	5400 / 7400	`Modules.RetailBanking.Web` + `Modules.RetailBanking`
`TflCbs.Host.Administration`	5500 / 7500	`Modules.Administration.Web` + `Modules.Administration`

All four set `Modules:Enabled = ["<domain>"]` — just the ONE domain. The core modules General/Reference/Authentication are `ICoreCbsModule` , so they register unconditionally and need no entry in the list. They run in shared-SSO mode (`DataProtection:KeyRing=TflOmniDb` + `Auth:SessionRevocation=TflOmniDb`) and set `ForwardedHeaders:Enabled` .

> **Canonical config lives in solution-level `config/` , owned by no host** (moved out of `TflCbs.Host.Main` on 2026-08-21). All five hosts `<Content Include="..\config\logmasking.json">` with `CopyToOutputDirectory="PreserveNewest"` , and load it from **`AppContext.BaseDirectory` , NOT the content root**. `logmasking.json` is the PII masking config, loaded `optional: false` because `MaskingOptions` defaults `Patterns` to EMPTY — a host missing it would log PAN/Aadhaar in free text unredacted with nothing to notice. Guarded by `ArchTests/LogMaskingConfigTests` .

`TflCbs.Gateway` — YARP reverse proxy. Dev `:5100 / :7100` .

- **Purpose.** Fronts all CBS hosts under ONE external origin: TLS termination, path-route

`/<Area>/*` to the owning host, load-balance/health-check, global rate limiting. One origin = one cookie domain = the precondition for cross-host SSO. **PURE PROXY** — no CbsFramework, no screens, NO project references.

- **Refs.** `Yarp.ReverseProxy` + OpenTelemetry only.

Layer: tests

Project	Kind	What it covers
`TflCbs.Tests`	xUnit, 149 tests	Cross-provider, non-destructive service tests across ALL nine service modules. (Name is historical — the `TflCbsServices` project is gone.) Also home to `SharedServiceHelperTests`, which fails the build on a privately re-declared `ServiceQuery` / `DbErrors` helper.
`TflCbs.ArchTests`	NetArchTest, 94 tests	Enforces architecture boundaries: framework ⊥ screens, namespace == assembly (`NamespaceDisciplineTests`), the promoted-module list (`ArchBoundaries.PromotedModules`), the `BoundaryGuaranteeTests` meta-guard, `ModuleConfigTests` (deploy config vs code), `LogMaskingConfigTests`, and resolve-all-modules. Refs the host, every `*.Web` RCL and every service module to inspect them.
`TflCbs.IntegrationTests`	xUnit + Mvc.Testing, 74 tests	Boots the REAL host pipeline in-process via `WebApplicationFactory` (`CbsWebAppFactory`) — middleware order included — and asserts runtime behaviour the unit tests cannot: the fail-closed access guard (`AccessGuardTests`), antiforgery (`AntiforgeryTests`) and authenticated access (`AuthenticatedAccessTests`). This is why `TflCbs.Host.Main/Program.cs` ends with `public partial class Program;`.
`TflCbs.E2E`	Playwright + TypeScript, 63 tests	**Outside the `.slnx`** (a Node project). Browser-level coverage: JavaScript behaviour, cookies, the strict CSP actually being enforced, and the reusable components working — 10 specs across a `guest` project (access guard, no credentials needed) and an `app` project (authenticated). `npx playwright test`.
`TflOmniLog.Tests`	xUnit	Unit tests for the logging library.

Layer: tools & demos

Project	Kind	What it does
`TflCbs.Demo`	console	Mirrors each service method across the nine modules against any provider. Kept in sync with Tests by the **cbs-sync-test-demo** agent.
`TflCbs.Tools.PasswordReset`	console	One-off admin tool for the Option A clean-break password migration: resets every `a_User` row to temp password `a`, hashed with PBKDF2 (`TflSecurityCrypto.PasswordHash`) with a unique per-user salt, writing `PASSWORDHASH` and stamping `PASSWORDVERSION = 1` via the typed `A_USER` entity. Supports `--dry-run`. Requires migration 0001 applied first. SQL Server target.
`TflCbs.Tools.DbMigrator`	console	The schema migration runner (DbUp). Applies the versioned DDL in `Migrations/<provider>/` exactly once each, in order, journalling to `CbsSchemaVersions`. Commands: `status` (read-only; exit 2 = pending), `preview`, `upgrade`, `baseline`. Reads the SAME `Database:*` config keys as every host. Deliberately references NO TflOmniDb / TflCbs.Entities / Framework: it must run against a database whose schema does not yet match the entity model. Runbook: `deploy/db-migrations.md`.

Not in the solution build

Tree	Why it is out
`TflCbs.Lab` (ns `TflCbs.Services.Lab`)	Holds EVERY converted proc; a distinct namespace avoids CS0433 collision with prod. Each proc is converted here first, then PROMOTED into the owning `TflCbs.Modules.<Domain>` / `TflCbs.Core.<Area>` assembly. Experimental, API-drifted.
`TflCbs.Lab.Demo`	Function-wise promotion harness (Roslyn extracts a single method from lab files) and the `perf` mode that times a legacy T-SQL proc against its migrated C# service.
`TflCbs.Tools.PerfProbe`	Measurement tool for the "DB-side filter/sort/page" workstream; run directly via `dotnet run`.
`libs/TflOmniDb/`, `libs/TflSecurityCrypto/`	The vendored in-house libraries (§3), each with its own solution, tests, demo and tooling. Built separately; consumed here as DLLs.
`TflCbs.E2E`	The Playwright suite — a Node project.
`codebase_shared/`	A **gitignored staging copy**: a self-contained tree for handing the Administration thin client to an outside team — the 3 projects they edit (`TflCbs.Modules.Administration{,.Web}`, `TflCbs.Host.Administration`) plus `TflCbs.Entities` and `TflCbs.Web.Shared` as source, and 9 platform assemblies as prebuilt DLLs in `refs/` (each with `.pdb` and `.xml`). This is why nine assemblies now set `GenerateDocumentationFile` — when an assembly ships as a DLL, IntelliSense is the only place its API semantics can travel. Repo-walking guards exclude it (`RepoFiles.Find`) so they don't see two of everything.

5. How they work together (composition)

The host's `Program.cs` (`TflCbs.Host.Main/Program.cs`) composes the app:

1. Register the legacy code-page provider (1252) — `TflSecurityCrypto`'s password crypto needs it or

login throws.

1. **READ THE GATE:** `Modules:Enabled` from config → `string[]` (`"*"`/absent = all).

`ModuleOn(serviceModule)` tests membership.

1. **CORE WEB PARTS ALWAYS ON:**

``` `csharp AddControllersWithViews() .AddApplicationPart(AuthenticationWebModuleMarker.Assembly) .AddApplicationPart(ShellWebModuleMarker.Assembly)` ```

1. **DOMAIN WEB PARTS GATED** by their backing SERVICE-module name:

``` `ModuleOn("General") -> Bank.Web marker (Area/service divergence) ModuleOn("HR") -> Hr.Web ModuleOn("Lockers") -> Lockers.Web ModuleOn("RetailBanking") -> RetailBanking.Web ModuleOn("Administration") -> Administration.Web` ```

1. **FRAMEWORK:** `builder.Services.AddCbsFramework(config)` (one call: session/cache/access

guard/search/DataAccess/reference cache/SSO).

1. **OBSERVABILITY:** `AddCbsObservability(config)` — OpenTelemetry traces + RED/runtime metrics over

OTLP. Config-gated on `Observability:Enabled`, OFF by default, so a local `dotnet run` emits nothing.

1. **LOGGING:** `builder.Host.ConfigureTflLogging()` + `AddTflLogging(config)`.
2. **SERVICE MODULES:** `builder.Services.AddCbsModules(config, <every module assembly by name>)`.

Discovery is **FULLY EXPLICIT** — there is no implicit anchor assembly any more, so every host must NAME every module it deploys: - Core (`ICoreCbsModule`, always registered): `GeneralModule`, `ReferenceModule`, `AuthenticationModule` - Domains (gated by `Modules:Enabled`): `LockersModule`, `HrModule`, `AccountsModule`, `ClearingModule`, `AdministrationModule`, `RetailBankingModule`

Each module's `AddServices` runs only if enabled, so controllers and their services toggle together. Unknown names throw at startup.

1. **PIPELINE, in order:** `UseCbsSecurityHeaders` (FIRST — CSP + X-Frame-Options + nosniff +

Referrer-Policy on every response, including static files and error pages) → `UseCbsForwardedHeaders` (honour the gateway's `X-Forwarded-Proto/Host` so the access guard's login returnUrl carries the GATEWAY origin; must precede `UseHttpsRedirection`) → exception handler + HSTS (non-dev) → `UseHttpsRedirection` → `UseStaticFiles` → `UseRouting` → `UseCbsFramework()` (login rate limiter, then session + fail-closed menu/auth guard; after routing, before endpoints) → `MapControllers()` (attribute routing pins legacy URLs) → `MapCbsHealthChecks()`.

Inside `UseCbsFramework` the rate limiter is FIRST: the `cbs-login` policy (registered by `AddCbsFramework`, `Security:LoginRateLimit`, per client IP + path, 10/60s) throttles the two credential POSTs — `/Account/Login` and `/Account/ChangePassword`, which opt in via `[EnableRateLimiting]` — so a throttled attempt is refused with 429 + `Retry-After` before the session is loaded, the identity cookie is decrypted or any DB work happens. It lives in the framework, so EVERY host has it, gateway or not; the gateway's own limiter is volumetric, and the DB lockout policy remains the per-account control.

The four thin hosts use the IDENTICAL `Program.cs` shape but register only their one domain marker, name only the module assemblies they deploy, and set `Modules:Enabled = ["<domain>"]`.

- Composition contracts: `TflCbs.Abstractions/ICbsModule.cs` + `ICoreCbsModule.cs`
- Discovery/registration: `TflCbs.Abstractions/ModuleRegistration.cs`

> **DEPLOYED-MODULE CROSS-CHECK.** `AddCbsModules` scans the output directory at startup and THROWS if a `TflCbs.Modules.<X>` / `TflCbs.Core.<X>` assembly is deployed but was not passed in. Nothing else catches that mistake: a host on `Modules:Enabled="*"` never names a module, so the unknown-name check cannot fire, and a missing `ICoreCbsModule` degrades SILENTLY to the framework's no-op ports (empty menu). Composition TESTS that build partial sets opt out with `verifyDeployedModules: false`.

---

6. Flow among the projects (a request, end to end)

SINGLE-HOST (default host `TflCbs.Host.Main`):

```
Browser
  | HTTP GET /Hr/State
  v
TflCbs.Host.Main (host)                    - security headers, routing
  v
```

```
CbsAccessMiddleware (Framework)           - fail-closed: authenticated?
  |                                         module selected? route in menu?
  v
StateController (Modules.Hr.Web RCL)      - bind, authorize screen, token
  v
StateService (TflCbs.Modules.Reference)   - business logic, returns Result<T>
  v
Repository<G_STATE> (TflOmniDb)           - typed CRUD
  v
Database (via DataAccess/DatabaseSelector) - SQL Server / Oracle / PostgreSQL
  ^
  | Result<T> flows back up
  v
Controller -> View (Razor, using shared partials from Web.Shared)
  v
Notify (Framework) -> _Notification partial -> Browser
```

Static assets css/js are served from `~/_content/TflCbs.Web.Shared/` and `~/_content/TflCbs.Modules.*.Web/`.

Note the `/Hr/State` example crosses the module boundary **as designed**: the SCREEN is an HR-area screen, but State is shared reference data, so the service comes from the Reference core module. Legacy menu placement is preserved on purpose (State sits under the HR menu) — menu modules and code modules are separate axes and are NOT realigned to match.

MULTI-HOST (gateway + thin hosts, with SSO):

```
Browser ---> TflCbs.Gateway (:7100, one origin)
              |  /Hr/*             -> TflCbs.Host.Hr (:7300)
              |  /Lockers/*        -> TflCbs.Host.Lockers (:7200)
              |  /RetailBanking/*  -> TflCbs.Host.RetailBanking (:7400)
              |  /Administration/* -> TflCbs.Host.Administration (:7500)
              |  everything else   -> TflCbs.Host.Main (default, :7225)
              v
```

Each host runs the SAME pipeline as above (each Area's `/_content/*` is routed to the owning host alongside its paths). Login happens ONCE on the central authority; the encrypted `CbsAuth` cookie (shared DataProtection key ring in the DB) is decrypted by whichever host the gateway routes to; that host rehydrates the session and reloads ITS OWN menu from the DB. Global logout is enforced via the shared revocation table. Details: `single-sign-on.txt`; port table: `port-map.md`.

---

7. Why each layer is separate (the reasoning)

Separation	Reason
Abstractions at the floor	One dependency-free assembly holds the contracts BOTH sides need (`Result`, `ICbsModule`, the ports, the shared query/error helpers), so the framework and the modules can cooperate without either referencing the other. It is also the one place a helper can live instead of being copy-pasted into every service.
Published contracts assembly	A consumer needing General's scroll takes `General.Contracts`, not General — so a cross-module dependency is a two-interface surface, not a whole domain.
Entities separate	One generated DB-shape contract, consumed as a compiled DLL so a schema regen doesn't force a source rebuild of every dependent's project graph.
Logging ASP.NET-free	So the zero-ASP.NET data layer can log without dragging in the web stack.
Services zero-ASP.NET	Business logic stays testable and host-agnostic; arch tests forbid it referencing ASP.NET.
Framework vs screens	Reusable plumbing evolves without touching 1,000 screens; screens can't reach past the framework, and the framework can't reach into a module.
Shared Web RCL	One copy of layout/partials/assets, served to every host via `_content`, so UI is consistent.
Web modules as RCLs	Each domain's screens build independently and can be composed into ANY host (full or thin).

Separation	Reason
Module per assembly	Bounded domains that self-register (`ICbsModule`), with the boundary enforced by the COMPILER rather than by folder discipline: one process + one DB in steady state, but any domain can be hosted alone and only a host that enables a module ships it.
Core vs domain modules	`ICoreCbsModule` = platform plumbing and shared masters that every composition needs (menu, bank variables, login, geography), so a thin host is genuinely standalone with a one-entry `Modules:Enabled` .
Thin hosts + gateway	Prove a module runs standalone and enable horizontal scale / independent deploy behind one origin — which is what makes cross-host SSO work.
Gateway as pure proxy	No business code, so it stays a thin, replaceable edge (TLS, routing, rate limit, health).
Libraries vendored, not linked	`libs/` puts `TflOmniDb` / `TflSecurityCrypto` source inside the repo boundary so a fresh clone builds, while the DLL-by-HintPath reference style keeps them substitutable and stops their transitive package graph leaking into every consumer.

---

8. Build order & conventions that fall out of this

- **One solution build orders everything.** `TflCbs.Entities` is referenced by bare-DLL HintPath, but

every consumer declares a `ReferenceOutputAssembly="false"` build-order edge, so CLI `dotnet build` and Visual Studio Rebuild All both sequence it correctly. No manual pre-build step.

- **Build the app with:** `dotnet build TflCbs.Host.Main` (build only). **STOP a running app**

(including one started by the VS debugger) before rebuilding to free DLL locks.

- **NAMESPACE == ASSEMBLY.** Every type sits under a namespace beginning with its own assembly name,

so a `using` names the assembly you are coupling to. Sanctioned exception: extension-method homes in `Microsoft.Extensions.DependencyInjection` / `Microsoft.AspNetCore.Builder` . Enforced by `NamespaceDisciplineTests` (which also forbids two assemblies sharing a namespace). No exceptions remain.

- **Every DEPLOYED module assembly must be handed to `AddCbsModules`** — see the cross-check in §5.

Adding a module means editing every host that deploys it, plus `ArchBoundaries.PromotedModules` .

- **Central Package Management:** add package VERSIONS in `Directory.Packages.props` ; csproj

`PackageReference` s are version-less.

- **Excluded from the `.slnx` :** `TflCbs.Lab` / `.Lab.Demo` (experimental, API drift) and

`TflCbs.Tools.PerfProbe` — build them standalone only. `libs/*` and `TflCbs.E2E` build with their own tooling.

- **Bare-DLL refs (`TflOmniDb` / `TflCbs.Entities` / `TflSecurityCrypto`) + ADO.NET drivers do NOT flow

transitively** — each project in the closure re-declares them.

- **No raw SQL / stored procedures for NEW master CRUD** — use `Repository<T>` .
- **Schema changes go through `TflCbs.Tools.DbMigrator`** — versioned DDL under

`Migrations/<provider>/` , applied once and journalled.

- **Arch boundaries are TESTED:** `dotnet test TflCbs.ArchTests` . Service behaviour:

`dotnet test TflCbs.Tests` . Runtime pipeline: `dotnet test TflCbs.IntegrationTests` . Browser behaviour: `npx playwright test` in `TflCbs.E2E` .

- After a service module's public method is added/changed/removed, run the **cbs-sync-test-demo**

agent to sync Tests + Demo.

---

9. Related docs

Doc	What it covers
`README.md` · `index.html`	The full documentation index
`architecture/reusable-boundary.md`	The horizontal framework/screen split
`architecture/modular-monolith.md`	The vertical business-layer split
`architecture/TrustBank-CBS-Platform-Architecture.md`	Client-facing platform architecture
`architecture/TrustBank-CBS-Architecture-Brief.md`	Management briefing note — position, risks, decisions
`guides/day-one.md`	Hands-on first-day setup for a new developer
`guides/building-a-crud-screen.md`	How to build one screen end-to-end
`guides/screen-definition-of-done.md`	The review gate for a migrated screen
`guides/starting-a-new-module.txt`	How to add a whole new domain module
`guides/search-and-rowtoken-flow.md`	The search + RowToken round trip
`port-map.md`	Every project's dev + deployed ports
`single-sign-on.txt`	Cross-host SSO
`deploy/db-migrations.md`	Versioned DDL and the migration runner
`observability/observability-primer.md`	Logs/metrics/traces + the dev backend
`faq-how-to.txt`	Day-to-day how-tos + troubleshooting
`FEATURES-SHOWCASE.txt`	One-line feature inventory
`../CLAUDE.md`	The project contract; **WINS on conflict**